Code Coverage Agent: Ablation Study

Knowledge injection + structured execution for AI-generated tests

Mark Pollack

March 2026

Abstract

We evaluate the effect of domain knowledge injection and structured execution on AI-generated test quality for Spring Boot projects. An AI agent (Claude Sonnet 4.6) is given a Spring Boot project with production code and no tests, and asked to write a comprehensive JUnit test suite. We compare five variants that progressively add capability: prompt quality, domain knowledge files, and a two-phase explore-then-act execution pattern. Each variant is evaluated by a four-tier automated judge cascade measuring build success, coverage improvement, practice adherence, and similarity to expert-written reference tests. On simple projects, a hardened prompt is the dominant lever. Knowledge injection and structured execution show no measurable benefit on trivial targets but are hypothesized to matter on complex, multi-layer applications.

Contents

1 Executive Summary **3**

2 Experimental Design **3**
- 2.1 Task . . . 3
- 2.2 Variants . . . 3
- 2.3 Scoring: Four Dimensions . . . 4
  - 2.3.1 Line Coverage (JaCoCo) . . . 4
  - 2.3.2 T3 Practice Adherence (LLM Judge) . . . 4
  - 2.3.3 Golden Test Similarity (AST Comparison) . . . 5
  - 2.3.4 Efficiency (Agent Behavior) . . . 5
- 2.4 Judge Cascade . . . 5
- 2.5 Agent Execution and Capture . . . 6
- 2.6 Dataset . . . 6

3 Results: Getting Started Guides **6**
- 3.1 Variant Comparison . . . 6
- 3.2 Per-Item Breakdown . . . 7
- 3.3 Growth Story . . . 8
- 3.4 Efficiency and Agent Behavior . . . 8
- 3.5 Cost Analysis . . . 9

4 Results: Pet Clinic **9**

5 Analysis **10**
- 5.1 Lever Ordering . . . 10
- 5.2 Coverage as a Ceiling . . . 10
- 5.3 Efficiency vs. Quality . . . 10
- 5.4 Limitations . . . 10

6 Discussion **11**

7 Conclusion **11**

Table 1: Variant comparison — Getting Started Guides (5 items, N=1 per cell)

Variant	Pass%	Cov%	T3	Golden	Eff	Cost
control	80%	85.7	0.61	0.62	0.83	$4.22
variant-a	100%	84.2	0.71	0.66	0.94	$4.06
variant-b	100%	95.6	0.76	0.74	0.93	$4.67
variant-c	100%	84.9	0.75	0.68	0.85	$5.13
variant-d	100%	94.1	0.7	0.66	0.75	$8.39

1 Executive Summary

Five variants progressively add capability: a naive prompt (control), a hardened prompt with testing guidance (variant-a), curated domain knowledge (variant-b), full knowledge base (variant-c), and a two-phase explore-then-act execution pattern (variant-d).

Key findings from Sweep 001 (5 Spring Getting Started guides, N=1 per cell):

1. A **hardened prompt** is the single biggest lever — control fails 1/5 items; all other variants pass 100%.
2. All improved variants perform **similarly on simple projects** — T3 adherence ranges 0.70–0.76, indistinguishable from noise at N=1.
3. Two-phase execution costs **2**× with no measurable benefit on simple targets.
4. Coverage is a **ceiling**, not a differentiator — 85–95% across all variants.
5. Golden similarity shows a **possible KB signal** on JPA and security items.

2 Experimental Design

2.1 Task

Given a Spring Boot project with production code and no tests, generate a comprehensive JUnit test suite targeting ≥80% line coverage. The agent operates in a fresh workspace with the project source, a prompt, and optionally a set of domain knowledge files. It has full tool access: file reading/writing, shell execution (compilation, test runs), and iterative repair.

The agent is Claude Sonnet 4.6. Single-phase variants (control through variant-c) are invoked via `AgentClient` from the Spring AI Claude Agent library, which provides a one-shot request–response pattern. Two-phase variant-d uses `ClaudeSyncClient` from the Claude Agent SDK Java directly, because `AgentClient` does not support session continuity — variant-d needs the explore phase's context preserved in memory for the act phase, which requires two turns within the same Claude Code session.

In all cases, each invocation is self-contained: the agent reads the project, writes tests, compiles, runs, checks coverage, and iterates until satisfied or the timeout expires.

2.2 Variants

The five variants form a progressive ablation — each adds one lever on top of the previous:

Table 2: Variant definitions

Variant	Prompt	Knowledge	Phase	What changes
control	v0-naive	none	single	Minimal instructions: "write tests for this project"
variant-a	v1-hardened	none	single	Detailed testing guidance: slice annotations, assertion patterns, iteration protocol
variant-b	v2-with-kb	3 KB files	single	Adds 3 curated knowledge files (coverage fundamentals, JaCoCo patterns, Spring test slices)
variant-c	v2-with-kb	full KB tree	single	Adds entire knowledge base (8 cheatsheets via index.md router)
variant-d	v3-explore + v3-act	full KB	two-phase	Two separate turns: first explore the project and KB, then write tests

2.3 Scoring: Four Dimensions

Each variant–item result is scored on four independent dimensions. Scores are **not combined** into a single number — they measure different aspects of test quality.

2.3.1 Line Coverage (JaCoCo)

Standard JaCoCo line coverage percentage. Measured deterministically by running `mvn clean test jacoco:report` on the agent's workspace after completion. Reports both baseline (always 0% since no tests exist initially) and final coverage.

Interpretation: Higher is better, but coverage alone does not measure test quality — a test that calls every method without assertions achieves high coverage with no value.

2.3.2 T3 Practice Adherence (LLM Judge)

An LLM-based judge (Claude Sonnet 4.6) evaluates the generated test suite against six criteria derived from Spring testing best practices:

1. **Test slice selection** — Does the test use the appropriate Spring test annotation (`@WebMvcTest`, `@DataJpaTest`, etc.) rather than `@SpringBootTest` everywhere?
2. **Assertion quality** — Are assertions specific and domain-aware (e.g., checking actual field values) rather than trivial (`assertNotNull`)?
3. **Error and edge case coverage** — Does the suite test validation failures, missing entities, boundary conditions?
4. **Domain-specific patterns** — Are Spring-specific testing patterns used correctly (MockMvc for controllers, TestEntityManager for JPA)?
5. **Coverage target selection** — Does the suite focus on behaviorally significant code rather than trivial getters?
6. **Version-aware patterns** — Does the suite use APIs appropriate for the detected Spring Boot version (Boot 4.x vs 3.x)?

Each criterion is scored 0.0–1.0 by the judge with supporting evidence. **T3** is the mean across all six criteria. The rubric is identical for all variants — knowledge files inform the agent's test authorship, not the judge's evaluation.

Interpretation: T3 measures *how well* the tests follow expert practices, independent of whether they compile or achieve coverage.

2.3.3 Golden Test Similarity (AST Comparison)

A deterministic comparison of the agent's test suite against expert-written reference ("golden") tests using JavaParser AST analysis. Five weighted dimensions:

- **Test method coverage** (weight 0.30) — ratio of reference test methods that have a corresponding agent test
- **Annotation alignment** (weight 0.20) — overlap in Spring test annotations used
- **Assertion style** (weight 0.20) — ratio of assertion count (agent vs reference)
- **Import alignment** (weight 0.15) — overlap in test imports (proxy for testing approach)
- **Injection pattern** (weight 0.15) — overlap in injected dependencies (`@MockitoBean`, `@Autowired`)

Interpretation: Golden similarity measures structural resemblance to expert tests. A score of 1.0 means the agent produced tests structurally identical to the reference. This is *not* an LLM judge — it is fully deterministic.

2.3.4 Efficiency (Agent Behavior)

Measures how efficiently the agent used its resources. Four weighted sub-metrics:

- **Build errors** (weight 0.35) — 1.0 − (failed builds/total builds). An agent that compiles correctly on every attempt scores 1.0.
- **Cost efficiency** (weight 0.20) — normalized inverse of total cost. Lower cost = higher score.
- **Recovery cycles** (weight 0.20) — 1.0 − (recovery iterations/total iterations). An agent that never needs to fix a broken build scores 1.0.
- **Tool utilization** (weight 0.25) — ratio of productive tool calls to total tool calls.

Interpretation: Efficiency measures the agent's operational quality — did it work cleanly, or did it thrash through compile-fix-compile cycles? Two agents can produce identical tests but differ dramatically in efficiency.

2.4 Judge Cascade

Judges are organized in a four-tier cascade. Lower tiers are binary gate checks; higher tiers produce continuous scores. If a lower tier rejects (e.g., build fails), higher tiers do not execute.

Tier 0: **BuildSuccessJudge** — does the project compile and all tests pass? (binary gate)

Tier 1: **CoveragePreservationJudge** — no regression from baseline coverage. (binary gate)

Tier 2: **CoverageImprovementJudge** + **GoldenTestComparisonJudge** — how much did coverage improve? How similar are the tests to expert reference tests? (continuous scores)

Tier 3: **TestQualityJudge** — LLM-based practice adherence across 6 criteria. (continuous score)

2.5 Agent Execution and Capture

Each agent invocation is captured via session log parsing (`SessionLogParser` from the agent-journal library). Both `AgentClient` and `ClaudeSyncClient` produce structured session logs that are parsed into a common `PhaseCapture` record. The captured exhaust includes:

- **Tool calls**: every file read, write, edit, glob, grep, and bash command
- **Token usage**: input tokens, output tokens, thinking tokens (extended thinking)
- **Turn count**: number of agent–environment round trips
- **Cost**: USD cost per invocation (API pricing)
- **Duration**: wall-clock time from prompt to final response
- **Phase count**: 1 for single-phase variants, 2 for variant-d (explore + act)

For two-phase variant-d, both phases run in the same `ClaudeSyncClient` session — `client.connect(explo` followed by `client.query(actPrompt)`. The explore phase's full context (tool results, reasoning) is preserved in the session for the act phase. Each phase's exhaust is captured separately via `SessionLogParser.parse()`.

2.6 Dataset

Sweep 001: 5 Spring Getting Started guides (bucket A — simple).

Table 3: Dataset items — Sweep 001

Item	Description	Classes	Complexity
gs-rest-service	Single REST controller + record	2	Trivial
gs-accessing-data-jpa	JPA entity + repository	2	Trivial
gs-securing-web	MVC + Spring Security config	3	Simple
gs-reactive-rest-service	Reactive REST handler	2	Trivial
gs-messaging-stomp-websocket	WebSocket STOMP controller	2	Simple

Each item has 1–3 golden reference test files written by domain experts, used for the golden similarity comparison.

Sweep 002 (planned): Spring Pet Clinic (bucket B — multi-layer, 15+ production classes, form-based MVC, JPA entity relationships, validation, Testcontainers integration tests, 17 golden reference test files).

3 Results: Getting Started Guides

3.1 Variant Comparison

All variants except control pass the binary gate checks (Tier 0–1) on all items. Control fails on `gs-messaging-stomp-websocket` (build failure). Coverage improvement is uniformly high (Tier 2) because the baseline is always 0% and the agent consistently achieves 80–100%.

The discriminating metrics are T3 practice adherence and golden similarity, which show variance across variants.

Table 4: Judge cascade results per variant (tier 0–3 + golden)

Judge	control	variant-a	variant-b	variant-c	variant-d
Build Success	100%	100%	100%	100%	100%
Coverage Preservation	100%	100%	100%	100%	100%
Coverage Improvement	100%	100%	100%	100%	100%
Practice Adherence	0.61	0.71	0.76	0.75	0.70
Golden Similarity	0.62	0.66	0.74	0.68	0.66

Table 5: T3 practice adherence per item (higher is better)

Item	control	variant-a	variant-b	variant-c	variant-d
gs-accessing-data-jpa	0.5	0.6	0.73	0.68	0.73
gs-messaging-stomp-websocket	0.45	0.65	0.55	0.5	0.5
gs-reactive-rest-service	0.73	0.82	0.82	0.9	0.73
gs-rest-service	0.67	0.82	0.88	0.88	0.88
gs-securing-web	0.72	0.68	0.83	0.78	0.67

3.2 Per-Item Breakdown

No single variant wins on all items. This is consistent with stochastic variation at N=1 rather than a systematic advantage for any variant.

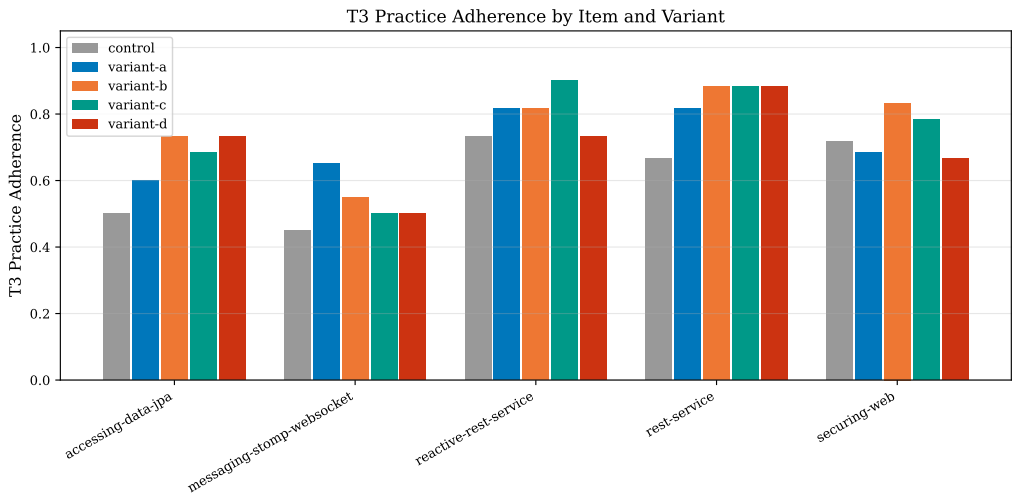

Figure 1: T3 practice adherence by item and variant. Per-item variance is high; no variant consistently dominates.

Table 6: Efficiency breakdown per variant (higher is better, except tokens/duration)

Variant	Build Err	Cost Eff	Recovery	Composite	Avg Out Tok	Avg Think Tok	Avg Duration
control	0.82	0.83	0.82	0.83	10,815	1,974	214s
variant-a	0.97	0.84	0.97	0.94	9,739	1,605	227s
variant-b	0.97	0.81	0.97	0.93	10,563	2,066	237s
variant-c	0.88	0.79	0.88	0.85	11,203	1,985	334s
variant-d	0.78	0.66	0.78	0.75	14,157	2,599	284s

3.3 Growth Story

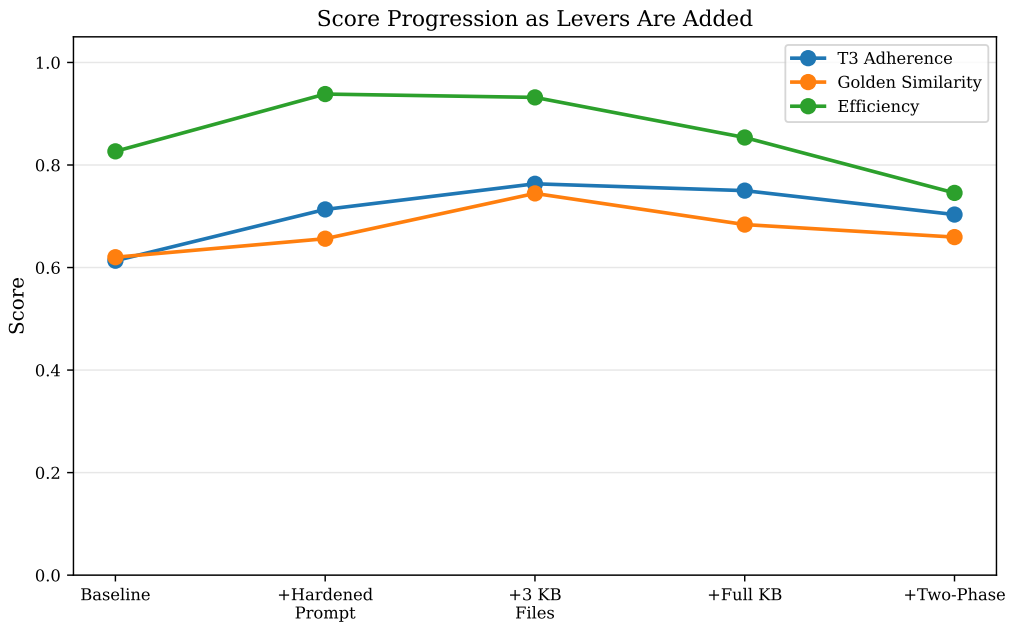

Figure 2: Score progression as levers are added. The jump from control to variant-a (hardened prompt) is the largest improvement. Subsequent levers show diminishing returns on simple projects.

3.4 Efficiency and Agent Behavior

Efficiency reveals a different story than quality scores. All single-phase variants achieve similar quality, but their operational behavior differs:

- **Build error rate**: variant-a scores highest (1.0 on most items) — the hardened prompt's compilation-first iteration protocol reduces wasted cycles. Control and variant-d both show more build failures.
- **Cost efficiency**: Single-phase variants cost \$4–5 per sweep. Variant-d costs \$8.39 (2×) because two Claude sessions are invoked per item. The explore phase adds context but also adds cost.
- **Token usage**: Variant-d generates the most output tokens (explore phase produces analysis text that is never directly used in tests). Knowledge-injected variants (b, c) use slightly more input tokens (reading KB files) but similar output tokens to variant-a.
- **Duration**: Single-phase variants complete in 2–5 minutes per item. Variant-d takes 5–10 minutes. Pet Clinic smoke test took 17 minutes for a single variant-a run — complex projects will need longer timeouts.

Table 7: Cost and token breakdown per variant

Variant	Total	Per Item	In Tok	Out Tok	Think Tok	Duration
control	$4.22	$0.84	109	54,075	9,869	1458s
variant-a	$4.06	$0.81	130	48,696	8,027	1527s
variant-b	$4.67	$0.93	137	52,816	10,330	1585s
variant-c	$5.13	$1.03	1,119	56,017	9,924	2098s
variant-d	$8.39	$1.68	1,190	70,783	12,997	1804s

3.5 Cost Analysis

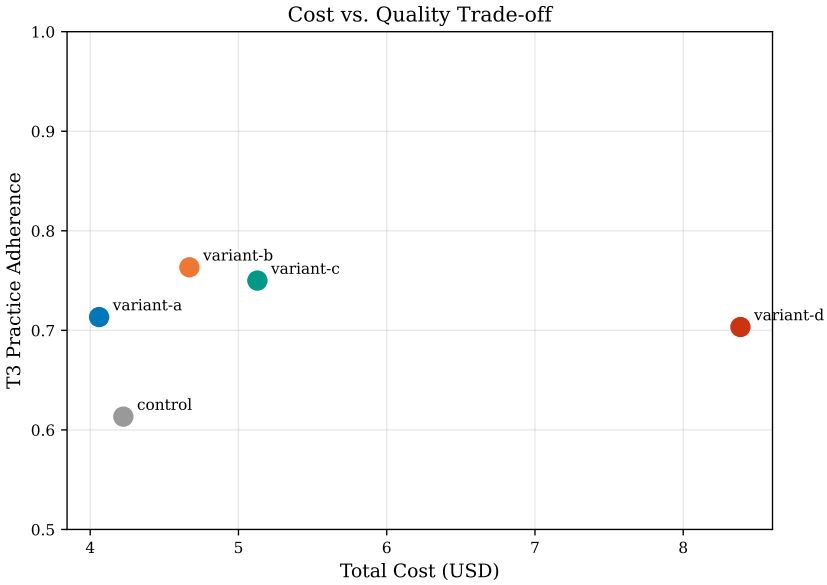

Figure 3: Cost vs. quality trade-off. Variant-d (two-phase) costs 2× without measurable quality improvement on simple targets. The efficient frontier is variant-a (best quality-per-dollar).

4 Results: Pet Clinic

Sweep 002 in progress — results pending.

Smoke test (variant-a on spring-petclinic):

- Line coverage: 93.6% (277/296 lines)
- T3 practice adherence: 0.87
- Golden similarity: 0.66
- Cost: $4.19
- Duration: 17 minutes (vs 2–3 minutes for guides)
- 60 test methods generated (reference has 59)

Full sweep (variant-a, variant-b, variant-d) will test whether knowledge injection and structured execution provide measurable benefit on a complex, multi-layer project.

5 Analysis

5.1 Lever Ordering

On simple projects, the dominant lever is prompt quality. The progression is:

1. Hardened prompt (+0.10 T3 over naive baseline)
2. Knowledge injection (+0.00–0.05 T3, within noise at N=1)
3. Two-phase execution (+0.00 T3, +100% cost)

The hypothesis is that this ordering changes on complex projects where the model cannot infer domain-specific testing patterns from context alone.

5.2 Coverage as a Ceiling

Coverage percentage is not useful for variant comparison on simple projects. Three of five items show identical coverage across all variants (85–95%). The projects are too simple — a single REST controller can be fully covered by 2–3 test methods. Coverage becomes more discriminating on larger projects (the Pet Clinic smoke test produced 93.6% from significantly more code).

5.3 Efficiency vs. Quality

The efficiency dimension reveals that agents can produce equivalent test quality through very different execution paths. Variant-a achieves the best quality-per-dollar: highest efficiency score (0.94) with competitive T3 (0.71) at the lowest cost ($4.06). Variant-d spends 2× and gets no measurable quality improvement on simple targets.

This suggests that on trivial tasks, additional "thinking time" (explore phase) and "reference material" (knowledge base) are wasted effort — the model already knows enough. The question is where the complexity threshold lies.

5.4 Limitations

- **N=1 per cell**: Every score is a single observation. Differences of ∼0.05 are indistinguishable from noise.
- **Simple dataset**: All items are single-layer, 1–3 class projects. Deliberately below the complexity threshold where knowledge should matter.
- **Same model**: All variants use Claude Sonnet 4.6. Cross-model comparison not yet tested.
- **No repeated trials**: Stochastic variation in LLM outputs means a single run may not represent typical behavior.
- **T3 is LLM-judged**: The practice adherence score comes from an LLM, which introduces its own noise. Golden similarity (AST-based) is deterministic but measures structure, not behavior.

6 Discussion

The central thesis is:

$$\textit{knowledge} + \textit{structured execution} > \textit{model alone}$$

Sweep 001 on simple projects provides the **null condition**: when the task is trivial, the model alone (with a good prompt) is sufficient. Knowledge injection adds cost without measurable benefit. The two-phase pattern adds both cost and complexity.

Sweep 001 is necessary but not sufficient to validate the thesis. The interesting question is whether results change on complex targets. Pet Clinic (15+ classes, 4 architectural layers, form-based MVC, JPA entity relationships, validation, Testcontainers) is designed to test exactly this.

If knowledge variants outperform on Pet Clinic but not on simple guides, the thesis holds: **knowledge investment is justified when task complexity exceeds what the model can infer from context alone**. If they don't, we learn that model capability has grown past the point where curated knowledge matters — at least for Spring Boot testing.

7 Conclusion

A hardened prompt is the minimum viable intervention for AI-generated test quality. On simple Spring Boot projects, additional knowledge injection and structured execution provide no measurable improvement at 2× the cost.

The experiment infrastructure — automated judge cascade, golden test comparison, efficiency tracking, and agent exhaust capture — is validated and ready for harder targets. Next: Spring Pet Clinic (complex target) and cross-model comparison (knowledge on a cheaper model vs. a more capable model without knowledge).